

# Week 1, Days 5-8

The roofline, inference arithmetic, and the KV cache as a serving engine sees it. Continues the Days 1-4 notes. Same model and card throughout: Qwen2.5-1.5B, fp16, one RTX 3060 12GB.

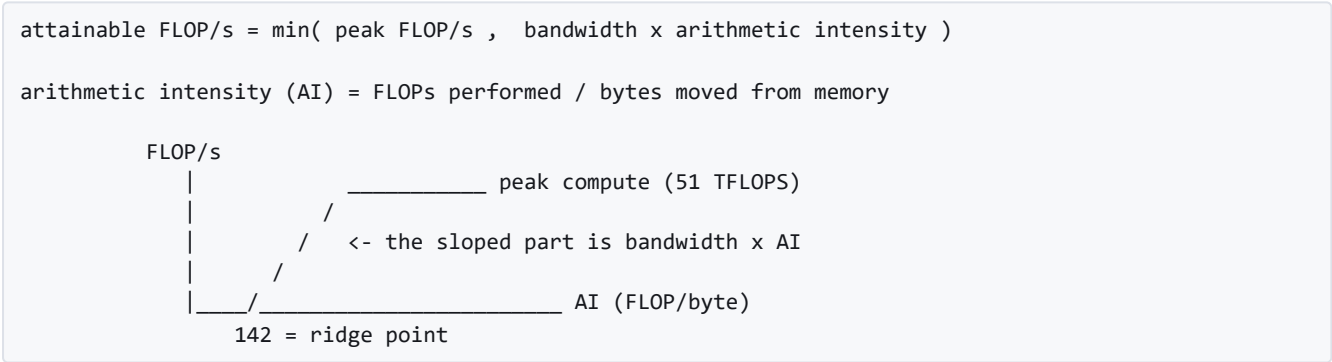
| Day | Topic                             | The one thing you must be able to derive                      |
|-----|-----------------------------------|---------------------------------------------------------------|
| 5   | Roofline and arithmetic intensity | Decode intensity is exactly 1 FLOP/byte, and the ridge is 142 |
| 6   | Inference arithmetic              | The parameter count summing to 1,543,714,304, and MBU vs MFU  |
| 7   | Buffer and rehearsal              | Nothing new. Say the four anchor answers out loud             |
| 8   | PagedAttention                    | The three wastes, and which one actually killed your run      |

**Correction carried in from Days 1-4.** Two numbers in the older docs/\*.md files are stale, and the CSVs are the authority. HF prefill is **122 ms (4205 tok/s)**, not 137 ms, and HF decode settled at **28.1 tok/s**, not 18-23. So the HF-to-vLLM decode gap is **2.84x**, not the ~4x written in the older prose. The Days 1-4 PDF has been re-rendered with these corrected.

## Day 5 · The roofline, and where your two phases sit on it

### 5.1 The model, in two lines

Any piece of work on a GPU is limited by one of two things: how fast the chip can do arithmetic, or how fast it can fetch the data to do arithmetic on. Plot achievable performance against arithmetic intensity and you get two straight lines and a corner.



Left of the ridge you are memory-bound and the only lever is moving fewer bytes. Right of it you are compute-bound and the levers are better kernels and tensor cores. The ridge point is a property of the hardware alone.

EXAMPLE 5-A: THE RIDGE POINT OF THIS CARD

ridge = peak FLOP/s / bandwidth = 51e12 / 360e9 = 141.7 ~= 142 FLOP/byte

Meaning: for every byte you pull from VRAM, you must do 142 floating point operations before the arithmetic units become the limit. Anything below that and the chip is waiting on memory.

### 5.2 The result worth memorising: decode has an intensity of exactly 1

This falls out cleanly and it is the cleanest single derivation in the week.

#### EXAMPLE 5-B: DECODE ARITHMETIC INTENSITY, BATCH 1

```
FLOPs per token    = 2 x params          (2 per weight: one multiply, one add)
bytes per token    = 2 x params          (fp16 is 2 bytes per weight)

AI = (2 x params) / (2 x params) = 1.0 FLOP/byte    exactly, for any fp16 model

with real numbers:
  FLOPs = 2 x 1,543,714,304          = 3.087e9 FLOP
  bytes = 2 x 1,543,714,304          = 3.087e9 B   = 3.09 GB
  AI    = 1.0                        vs a ridge of 142 -> 142x memory-bound
```

The dtype cancels. In int8 it is 2 FLOP/byte, in int4 it is 4. That is exactly why weight-only quantisation is a bandwidth play and not a compute play: it does not make the maths faster, it makes there be fewer bytes in front of the same maths.

### 5.3 Batching moves you along the axis, and you can compute where the knee should be

Add a second sequence to the batch. The weights are read once and serve two rows, so the bytes stay fixed and the FLOPs double.

#### EXAMPLE 5-C: INTENSITY AS A FUNCTION OF BATCH SIZE

```
AI(B) = (2 x params x B) / (2 x params) = B FLOP/byte

B = 1    AI = 1    memory-bound, tensor cores nearly idle
B = 64   AI = 64   still left of the ridge
B = 142  AI = 142  the ridge. compute becomes the limit
B = 256  AI = 256  compute-bound

so the predicted knee is at batch ~142
```

**Measured knee: 64.** The prediction is high, and the reason is that the derivation above ignores the KV cache. KV bytes do *not* amortise across the batch: each sequence has its own cache and each one must be read. So the true denominator grows with B, real intensity is below B, and the ridge arrives earlier than the clean formula says. Being able to state both the clean prediction and the reason it overshoots is a stronger answer than either number alone.

### 5.4 Prefill sits on the other side, for the same reason

#### EXAMPLE 5-D: PREFILL INTENSITY, 512-TOKEN PROMPT

```
AI = (2 x params x 512) / (2 x params) = 512 FLOP/byte    vs ridge 142
512 / 142 = 3.6x past the ridge -> compute-bound

and the threshold: a prompt shorter than 142 tokens is MEMORY-bound prefill
```

Prefill and decode are the same operation at two different values of one variable. That single sentence explains the whole 150x throughput asymmetry, and it also explains why a serving engine cares so much about batching short requests together.

### 5.5 Theoretical and achievable bandwidth, and why you must say which one

| Figure      | Value                           | Source                                             | Use it for                            |
|-------------|---------------------------------|----------------------------------------------------|---------------------------------------|
| Theoretical | 360.0 GB/s                      | NVML: 7501 MHz over a 192-bit bus                  | The ridge point, and nothing else     |
| Achievable  | 291.5 GB/s (81% of theoretical) | scripts/measure_bandwidth.py, large streaming read | Every efficiency percentage you quote |

The 19% shortfall is DRAM refresh, row activation, and the fact that no real access pattern is perfectly sequential. Quoting a percentage against the theoretical figure makes your own engine look worse than it is.

#### EXAMPLE 5-E: THE SAME RUN, TWO DENOMINATORS

```
vLLM decode: 247 GB/s achieved
  against 291.5 achievable -> 85% MBU      <- the honest number
  against 360.0 theoretical -> 69% MBU     <- what docs/baseline-vllm-results.md says

decode ceiling in tok/s:  291.5 / 3.09 = 94 tok/s      (achievable)
                        360.0 / 3.09 = 117 tok/s     (theoretical, unreachable)
```

### 5.6 Pope et al., and what to take from it

[Efficiently Scaling Transformer Inference](#) is the one paper to read twice this week. What it gives you that nothing else does: it treats inference as a partitioning problem with an explicit cost model, derives which parallelism layout wins in which regime, and it is the origin of the latency-versus-throughput framing that every serving paper since has assumed. For an academic interview it is the paper that shows you think in cost models rather than in benchmarks.

**Sources:** [Pope et al., Efficiently Scaling Transformer Inference](#) · [Williams et al., Roofline](#) · [Horace He, Making Deep Learning Go Brrrr](#) · [kipply, Transformer Inference Arithmetic](#)

## Day 6 · Inference arithmetic you can redo on a whiteboard

### 6.1 Counting the parameters, and hitting the published total exactly

This is a standard interview exercise. It is not hard, it is just bookkeeping, and getting it right in front of someone proves you know what is actually in the model.

#### EXAMPLE 6-A: QWEN2.5-1.5B, SUMMED TO THE PUBLISHED COUNT

```
Embedding (tied input/output, counted once)
151,936 x 1,536 = 233,373,696

Per layer:
q_proj 1536 x 1536 + bias 1536 = 2,360,832
k_proj 1536 x 256 + bias 256 = 393,472 (2 KV heads x 128 = 256)
v_proj 1536 x 256 + bias 256 = 393,472
o_proj 1536 x 1536, no bias = 2,359,296
gate_proj 1536 x 8960 = 13,762,560
up_proj 1536 x 8960 = 13,762,560
down_proj 8960 x 1536 = 13,762,560
2 x RMSNorm weight (1536 each) = 3,072
per layer = 46,797,824
x 28 layers = 1,310,339,072
+ embedding = 233,373,696
+ final RMSNorm = 1,536
TOTAL = 1,543,714,304 matches exactly

fp16 bytes = 1,543,714,304 x 2 = 3,087,428,608 B = 2944.40 MiB
measured resident on the card = 2945.29 MiB (0.89 MiB over)
```

Two things to say out loud from that table:

- **The MLP is 88% of each layer** (41.29M of 46.80M), attention is 11.8%. Most of the weight traffic is MLP. Almost all of the systems complexity is attention, because attention is the part whose cost grows with sequence length.
- **The embedding is 15.1% of the model** and does zero arithmetic. Pure lookup.

The 0.89 MiB overshoot is RoPE `inv_freq` buffers plus allocator rounding. Knowing why your measurement is slightly above the floor, rather than hand-waving it, is the difference between a measurement and a number.

### 6.2 The 2 x params per token rule, derived and bounded

Every weight in a linear layer is used exactly once per token: one multiply, one add. Two floating point operations per parameter. That is the whole derivation.

#### EXAMPLE 6-B: CHECK THE RATIO ON ONE REAL PROJECTION

```
q_proj at batch 1, one token: [1, 1536] x [1536, 1536]
FLOPs = 2 x M x K x N = 2 x 1 x 1536 x 1536 = 4,718,592
params = 1536 x 1536 = 2,359,296
ratio = 4,718,592 / 2,359,296 = 2.000 exactly, as claimed
```

**State the caveat before you are asked.** The rule counts linear layers only. It ignores the attention score computation, which grows as  $\text{seq}^2$  while the linear term grows as  $\text{seq}$ . At 512 tokens attention adds about 3% and the rule is fine. Past a few thousand tokens it undercounts, and past tens of thousands it is wrong.

6.3 MBU and MFU: pick the one that matches your bound

|            | MFU (model FLOPs utilisation)                  | MBU (model bandwidth utilisation)             |
|------------|------------------------------------------------|-----------------------------------------------|
| Definition | achieved FLOP/s / peak FLOP/s                  | achieved GB/s / achievable GB/s               |
| Use for    | Prefill, training, large batches               | Decode at small batch                         |
| Why        | Compute-bound, so FLOPs is the scarce resource | Memory-bound, so bytes is the scarce resource |

EXAMPLE 6-C: BOTH METRICS ON YOUR OWN RUNS

```
PREFILL, 512 tokens, MFU
FLOPs    = 2 x 1.544e9 x 512 = 1.58e12
HF       122 ms -> 12.99 TFLOPS / 51 = 25.5% MFU
vLLM    124 ms -> 12.78 TFLOPS / 51 = 25.1% MFU

DECODE, batch 1, MBU (bytes = 3.09 GB per token, denominator 291.5 GB/s)
HF       28.1 tok/s -> 35.6 ms -> 86.8 GB/s -> 29.8% MBU
vLLM    79.9 tok/s -> 12.5 ms -> 247.0 GB/s -> 84.7% MBU
```

Reporting decode as MFU would be a category error and an interviewer will notice. Decode at batch 1 runs at roughly 3.087e9 FLOP / 0.0125 s = 0.25 TFLOPS, which is 0.5% of the compute peak. That number is true and completely uninformative, because compute was never the constraint.

6.4 The latency-throughput Pareto, measured

The two things a serving system is asked for pull against each other. Latency is per-token time for one user (TPOT). Throughput is tokens per second across all users. Batching buys the second at the cost of the first, and the useful question is where the exchange rate turns bad.

EXAMPLE 6-D: YOUR VLLM BATCH SWEEP, REALISTIC POOL

```
throughput derived as batch / TPOT, from results/vllm_batch_sweep.csv

batch    TPOT ms    throughput tok/s    what is happening
  1       11.28         88.6    memory-bound, weights read for one row
 16       17.13        934    still nearly free: 15x work, 1.5x latency
 32       21.31       1502
 64       23.73       2697    <-- knee. 30x throughput for 2.1x latency
128       44.30       2889    +7% throughput for +87% latency
256       86.29       2967    +10% throughput for +264% latency
```

**How to say it.** From batch 1 to 64 throughput rose 30x while per-token latency rose only 2.1x, because the weight read was already being paid and batching just spread it over more rows. Past 64 the tensor cores are saturated, so extra sequences buy almost no throughput and cost latency linearly. If the SLO is a TPOT budget, 64 is the answer on this hardware and this model.

Sources: kipply, Transformer Inference Arithmetic · Databricks, LLM Inference Performance Engineering (MBU) · Pope et al.

## Day 7 • Buffer and rehearsal

No new material. This day exists because plans slip, and because the failure mode of a three-week plan is finishing the reading and being unable to say any of it out loud.

### 7.1 The rehearsal protocol

For each item below: close the notes, say it out loud, time it, and stop at 90 seconds. If you cannot, do not re-read. **Re-derive it from a blank page.** Re-reading produces recognition, which is not what an interview tests.

### 7.2 The six derivations from Week 1

| # | Derive, unaided                                               | Lands on                                                 |
|---|---------------------------------------------------------------|----------------------------------------------------------|
| 1 | Why attention scores are divided by <code>sqrt(d_head)</code> | Variance d, std 11.3 at head_dim 128, softmax saturation |
| 2 | The KV cache size formula, for this model                     | 28 KiB/token, from 2 KV heads                            |
| 3 | RoPE's score depends only on <code>m - n</code>               | 0.90930 both ways                                        |
| 4 | The ridge point and where the two phases sit                  | 142 FLOP/byte; decode 1, prefill 512                     |
| 5 | The parameter count                                           | 1,543,714,304, summed not quoted                         |
| 6 | Prefill vs decode asymmetry                                   | 4205 vs 28.1 tok/s = 150x                                |

### 7.3 The four anchor answers

Almost any question in this field routes back to one of these. Two of them are already fully supported by Week 1 material; two arrive in Week 2.

1. **The KV-cache OOM story** (ready now). 123,565 tokens, 62 KiB/token measured against 28 predicted, `torch.cat` copy-on-grow, then external fragmentation with 1.3 GiB free inside the pool.
2. **The HF-vs-vLLM profiling result** (ready now). 28.09 ms/token saved, 25.34 ms of it idle time removed (90%), 2.60 ms faster kernels (9%), launches per step 1198 to 17.
3. **The batching knee** (numbers ready, framing in Week 2 Days 5-6). Knee at 64: 30x throughput for 2.1x latency, then the exchange rate collapses.
4. **The FlashAttention derivation** (Week 2 Days 2-4). The one anchor the repo cannot support with a measurement, so the derivation itself has to carry it.

### 7.4 Known weak spots, named on purpose

- No hand-written CUDA kernel yet. Reading fluency was the Week 1 goal, not authorship. The honest line is "I can read a kernel and tell you why it is slow; I have not shipped one."
- The batching knee is measured on one GPU and one 1.5B model. Do not generalise it to a 70B model out loud.
- The two-box variance result (two nominally identical 3060s, 1.22x apart, cause unexplained) is a good answer to "tell me about a measurement you could not explain" and a bad thing to be surprised by.

## Day 8 · PagedAttention: the KV cache as a memory manager problem

### 8.1 The setup

You already have the size formula from Day 2: 28 KiB per token for this model. Day 8 is about what a serving engine does with that number when many sequences are alive at once and none of them has told you how long it will be.

The naive approach, which is what HuggingFace does, is one contiguous buffer per sequence. That forces a choice at admission time: how big? You do not know the output length in advance, so you reserve for the maximum. Everything that goes wrong follows from that one decision.

### 8.2 The three wastes

The PagedAttention paper names three, and they are distinct. Being able to separate them is the point of this day.

| Waste                         | What it is                                                                                                                                              | Where it shows in your run                                                                                                                                                         |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Reservation</b>            | Memory reserved for tokens the sequence will eventually generate but has not yet. Held, unusable by anyone else, for the whole lifetime of the request. | Not visible in your HF sweep, because you ran one sequence and grew it organically. It is the dominant waste in real multi-request serving.                                        |
| <b>Internal fragmentation</b> | Memory inside the reserved block that is never used, because the sequence stopped short of the maximum length it was sized for.                         | Also not visible at batch 1. Under a naive engine with <code>max_model_len 784</code> and a sequence that stops at 100 tokens, 87% of that sequence's reservation is dead.         |
| <b>External fragmentation</b> | Free memory that exists but is broken into gaps, none large enough for the contiguous block being requested.                                            | <b>This is literally your crash.</b> OOM at 10,437 MiB allocated with 11,764 MiB reserved: 1.3 GiB free inside the pool, no single gap big enough for the next $2 \times$ KV slab. |

**The sentence that makes this yours.** "External fragmentation is not a concept I read about, it is the specific failure I watched. The card had 1.3 GiB free and still raised `OutOfMemoryError`, because `torch.cat` needed one contiguous allocation and the free space was scattered between blocks the allocator had already parked."

### 8.3 The mechanism: block tables

PagedAttention borrows the idea that operating systems use for virtual memory. Stop requiring the KV cache of a sequence to be contiguous. Instead:

1. Cut the KV memory into fixed-size **blocks**, each holding a fixed number of tokens. vLLM used 16 tokens per block in your run.
2. Give every sequence a **block table**: a list mapping its logical token positions to the physical blocks that actually hold them.
3. When a sequence needs room for token 17, allocate *one* free block from the pool and append it to that sequence's table. No copy, no contiguity requirement, no growth of anything already written.
4. The attention kernel is rewritten to read through the block table instead of striding a flat array.

All three wastes go away at once. Reservation waste goes because you allocate a block at a time instead of a maximum-length buffer. Internal fragmentation is bounded to at most one block per sequence (at most 15 wasted token slots out of 16). External fragmentation is impossible because every block is the same size, so any free block fits any request.

#### EXAMPLE 8-A: THE POOL IN YOUR OWN VLLM RUN

```
from the engine init log and results/vllm_batch_sweep.csv:
num_gpu_blocks    = 14,836 blocks
block size        =      16 tokens
pool capacity     = 14,836 x 16 = 237,376 tokens
pool bytes        = 237,376 x 28 KiB = 6.34 GiB           matches the log

compare against the HF path on the identical card:
HF    died at 123,565 tokens of one sequence
vLLM held    237,376 tokens of KV, and never OOMs this way at all

ratio = 237,376 / 123,565 = 1.92x, which is the 2x that torch.cat was costing
```

That 1.92x is worth pausing on. It is almost exactly the copy-on-grow factor from Day 2. The paged engine fits nearly twice the context on the same card, and the reason is not a better kernel or a smaller cache. It is that it never pays for the same token twice.

### 8.4 Why vLLM has no OOM curve at all

A detail that makes the experimental design defensible. vLLM reserves the entire KV pool as one slab at startup, sized by `gpu_memory_utilization` (0.9 in your run). So device memory is *flat* for the whole sweep. There is no rising curve and no crash. When the pool fills, vLLM does not fail, it **schedules fewer sequences and queues the rest**.

**The methodological point.** The failure mode moved from a crash to a queue. To observe the KV wall in vLLM you read block occupancy and queue depth, not `nvidia-smi`. This is why the OOM experiment had to be run on HuggingFace: HF grows organically, so the crash is a real memory event you can plot. That contrast is the result, not a limitation of the setup.

### 8.5 Prefix caching and RadixAttention, in one paragraph

Blocks are immutable once written, so two sequences that share a prefix can point at the *same* physical blocks instead of each holding a copy. Reference counting handles the sharing, and copy-on-write handles the point where they diverge. vLLM exposes this as prefix caching. RadixAttention (SGLang) goes further: it keeps completed requests' blocks in a radix tree keyed by token prefix, so a later request that shares a prefix with an earlier one skips that much prefill entirely. This matters enormously for system prompts, few-shot examples, and multi-turn chat, where the shared prefix is most of the input. Note that this is a *prefill* optimisation, not a decode one, so it moves TTFT rather than TPOT.

Your vLLM baseline explicitly disabled prefix caching, so the second run could not reuse the first run's prefill. That is why the measurement is honest.

### 8.6 Exit bar for Days 5-8

1. Draw the roofline, mark the ridge at 142 FLOP/byte, and place decode (AI 1) and prefill (AI 512) on it.
2. Derive AI = 1 for fp16 decode in two lines, and say why int4 makes it 4.
3. Predict the batching knee at 142 from  $AI(B) = B$ , then explain why the measured knee is 64.
4. Sum the parameter count to 1,543,714,304 without notes.
5. State MBU vs MFU and give one measured value of each from your own runs.
6. Name the three wastes and say which one caused your OOM, with the two memory figures.

### 8.7 Self-test

#### Q1. Estimate the memory bandwidth needed to serve this model at 100 tok/s at batch 1.

Each token reads the full weight set, 3.09 GB. 100 tok/s therefore needs 309 GB/s of achievable bandwidth. This card achieves 291.5 GB/s, so 100 tok/s is not reachable at batch 1 on this hardware. The ceiling is 94 tok/s, and vLLM's measured 79.9 is 85% of it.

#### Q2. Why did quantising to NF4 make your decode slower, when quantisation is a bandwidth play?

Because `bitsandbytes` dequantises the weights back to fp16 before the GEMM. The bytes are smaller on the card (2945 to 1099 MiB) but the dequantisation happens on the critical path and the matmul still consumes fp16. So no bandwidth is saved where it counts, and the extra work costs time: 28.1 down to 12.4 tok/s. The lesson is that a quantisation win requires a kernel that consumes the quantised format directly.

### **Q3. Your engine reserves 6.34 GiB of KV at startup and never releases it. Is that wasteful?**

No, it is the opposite. A fixed pool of uniform blocks makes external fragmentation impossible and makes admission control a counting problem rather than an allocation gamble. The cost is that the memory cannot be used for anything else, which is fine because on an inference server there is nothing else. The alternative, organic growth, is what fragmented and crashed at 123,565 tokens.

### **Q4. Why does batching help so much at batch 1 to 16, and so little at 128 to 256?**

Because arithmetic intensity at batch B is roughly B FLOP/byte and the ridge is 142. Below the ridge the weight read is already paid and extra sequences ride along free, so throughput scales almost linearly while latency barely moves. Above it the tensor cores are the constraint, so each added sequence costs proportional compute. Measured: 30x throughput for 2.1x latency up to batch 64, then 10% more throughput for 264% more latency out to 256.

**Sources:** [Kwon et al., Efficient Memory Management for LLM Serving with PagedAttention](#) · [Zheng et al., SGLang / RadixAttention](#) · [vLLM launch post](#) · [Pope et al.](#)

---

Repo sources for every measured number above: `results/baseline_hf.csv`, `results/baseline_vllm.csv`, `results/oom_sweep.csv`, `results/vllm_batch_sweep.csv`, `results/profile_summary.csv`, `docs/profiling.md`, `docs/oom-results.md`. Where a CSV and a prose doc disagree, the CSV is used.